Table des matières

Cours 04 — Chargement, déchargement et connectivité 1
Objectifs 1
Théorie 1
QCM — style examen (Cours 04) 6
Points clés à retenir 10

Cours 04 — Chargement, déchargement et connectivité

Module 4/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

Comment lire ce module — Le domaine 3 pèse 18 %. Il répond à une question très concrète : comment la donnée entre dans Snowflake, et comment elle en ressort. Deux grands temps : le chargement par lots (des fichiers → un *stage* → `COPY INTO` → une table) et l'ingestion continue (Snowpipe, Streams, Tasks, Dynamic Tables). On finit par la connectivité (pilotes, connecteurs, API). Le lien avec le Cours 01 : `CREATE TABLE` y fabriquait la coquille vide ; ici, on la remplit.

Objectifs

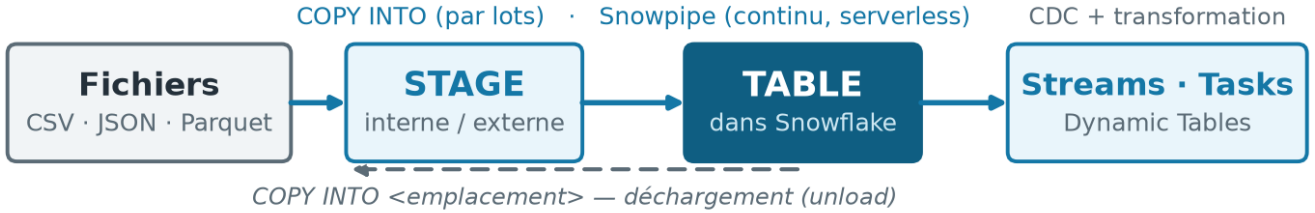
À la fin de ce module, vous saurez :

- situer les stages, internes (utilisateur, table, nommé) et externes (S3/Azure/GCS), et le rôle d'une *storage integration* ;
- définir un format de fichier (CSV, JSON, Parquet, Avro, ORC, XML) ;
- charger avec `COPY INTO <table>` : gestion d'erreurs (`ON_ERROR`), validation, idempotence ;
- décharger avec `COPY INTO <emplacement>` ;
- distinguer les modes d'ingestion continue : Snowpipe (+ *streaming*), Streams, Tasks, Dynamic Tables ;
- reconnaître les connecteurs, pilotes et API qui relient un programme à Snowflake.

Théorie

1. Le parcours de la donnée — vue d'ensemble

© 2026 iAlexMG. Tous droits réservés.



Le pipeline : des fichiers déposés dans un stage, chargés par `COPY INTO` (par lots) ou par Snowpipe (en continu) vers une table ; les Streams, les Tasks et les Dynamic Tables orchestrent la suite ; `COPY INTO <emplacement>` fait le chemin inverse, le déchargement.

Charger, c'est amener des fichiers (souvent du stockage objet du cloud) dans une table Snowflake. Le trajet type comporte trois pièces :

1. un stage — l'aire d'atterrissage où vivent les fichiers ;
2. un format de fichier — la grammaire pour les lire (délimiteur, en-tête, compression…) ;
3. une commande `COPY INTO` — qui lit les fichiers du stage et écrit les lignes dans la table.

Deux grandes façons de charger : par lots (*bulk*, avec `COPY INTO` sur un entrepôt) et en continu (Snowpipe & co., à mesure que les fichiers arrivent). On voit les deux.

2. Les stages — où atterrissent les fichiers

Labo : `Labo_04_SNOW.md`, étape 1.

Un stage est un emplacement de fichiers que Snowflake sait lire/écrire. Deux familles :

- Stages internes — le stockage géré par Snowflake. Trois variantes :
  - stage utilisateur (`@~`) — un par utilisateur, pour ses fichiers perso ;
  - stage de table (`@%ma_table`) — attaché à une table précise ;
  - stage nommé interne (`@mon_stage`) — un objet qu'on crée (`CREATE STAGE`), partageable et réutilisable. Le plus courant.
- Stages externes — un pointeur vers un *bucket* que vous possédez (S3, Azure Blob, GCS). Le stage ne stocke rien : il référence votre stockage cloud.

La *storage integration* — La bonne façon de brancher un stage externe. Plutôt que de coller vos clés d'accès cloud en clair dans le SQL, vous créez une intégration de stockage : un objet (créé par ACCOUNTADMIN) qui range les identifiants de façon sécurisée et délègue l'accès au bucket. Le stage y fait référence, et aucun secret ne traîne dans le code. *(Reconnaître.)*

© 2026 iAlexMG. Tous droits réservés.

Pour déposer un fichier local dans un stage interne, on utilise `PUT` (via SnowSQL ou le connecteur, pas depuis un worksheet) ; `GET` fait l'inverse (télécharger). Dans Snowsight, l'assistant « Load Data » (glisser-déposer) remplace le `PUT` pour les petits fichiers.

3. Les formats de fichier

Labo : `Labo_04_SNOW.md`, étape 1.

Un format de fichier (`FILE FORMAT`) décrit comment lire les fichiers. Snowflake gère le structuré délimité et le semi-structuré :

Type	Exemples d'options
CSV (délimité)	`FIELD_DELIMITER`, `SKIP_HEADER`, `FIELD_OPTIONALLY_ENCLOSED_BY`, `NULL_IF`
JSON	`STRIP_OUTER_ARRAY` (éclater un tableau JSON en lignes)
Parquet / Avro / ORC	formats colonnes binaires, courants en *data lake*
XML	balisage

On peut le définir à la volée dans le `COPY`, ou le nommer (`CREATE FILE FORMAT`) pour le réutiliser. *(Astuce examen : `INFER_SCHEMA` peut deviner les colonnes d'un fichier Parquet ou CSV pour vous éviter d'écrire le `CREATE TABLE` à la main.)*

4. `COPY INTO <table>` — le chargement par lots

Labo : `Labo_04_SNOW.md`, étapes 3, 4 et 5.

C'est la commande de chargement. Elle lit les fichiers d'un stage et insère les lignes :

```
COPY INTO ma_table
  FROM @mon_stage
  FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1)
  ON_ERROR = 'CONTINUE';
```

Trois réflexes que l'examen teste :

- Gestion d'erreurs — `ON_ERROR` dit quoi faire d'une ligne invalide :
  - `ABORT_STATEMENT` (défaut) — tout annuler à la première erreur ;
  - `CONTINUE` — sauter les lignes fautives, charger le reste ;
  - `SKIP_FILE` (ou `SKIP_FILE_n`) — sauter le fichier entier dès *n* erreurs.

© 2026 iAlexMG. Tous droits réservés.

- Validation à blanc — `VALIDATION_MODE` : `RETURN_ERRORS` (ou `RETURN_n_ROWS`) teste le chargement sans rien écrire, parfait pour vérifier un fichier avant de l'ingérer.
- Idempotence : `COPY` retient les fichiers déjà chargés (métadonnées de charge, ~64 jours) et les ignore au prochain appel. Rejouer un `COPY` ne double donc pas les données, sauf `FORCE = TRUE`, qui recharge tout.

`COPY` peut aussi transformer légèrement à la volée (choisir/réordonner des colonnes, `CAST`, `SUBSTR`) via un `SELECT` sur le stage. Pour de la transformation lourde, on charge d'abord, on transforme ensuite (Cours 05).

5. `COPY INTO <emplacement>` — le déchargement

Labo : `Labo_04_SNOW.md`, étape 2.

La même commande, inversée : exporter le résultat d'une requête vers un stage (fichiers) :

```
COPY INTO @mon_stage/export/
  FROM (SELECT * FROM ma_table)
  FILE_FORMAT = (TYPE = CSV HEADER = TRUE)
  OVERWRITE = TRUE;
```

Snowflake écrit un ou plusieurs fichiers (souvent compressés, découpés en morceaux). On récupère ensuite les fichiers avec `GET` (ou depuis le bucket externe). C'est le déchargement (*unloading*).

6. L'ingestion continue — au-delà du lot

Labo : `Labo_04_SNOW.md`, étapes 6 et 7.

Charger « quand un lot arrive » ne suffit pas toujours ; parfois la donnée coule en permanence. Quatre objets à distinguer nettement :

- Snowpipe — le chargement continu, serverless : dès qu'un fichier arrive dans le stage, Snowpipe le charge en micro-lots, sans entrepôt (facturé au compute serverless). Déclenché par des notifications du cloud (*auto-ingest*) ou par appel REST.
- Snowpipe Streaming — pour insérer des lignes (pas des fichiers) à très faible latence, via une API, idéal pour du *streaming* type Kafka.
- Streams — un objet de capture de changements (*CDC*) : il suit les `INSERT`/`UPDATE`/ `DELETE` d'une table depuis la dernière lecture. On lit le stream pour savoir ce qui a changé.
- Tasks — un objet qui exécute du SQL sur un horaire (`SCHEDULE`) ou en chaîne, chaque tâche attendant la précédente : c'est ce qu'on appelle un DAG (*directed acyclic graph*, un graphe orienté acyclique, autrement dit un enchaînement qui ne revient jamais sur ses pas). Souvent couplé à un Stream : « toutes les 5 min, traite ce qui a changé ».
- Dynamic Tables — la version déclarative : vous écrivez la requête cible, Snowflake maintient la table à jour tout seul selon un `TARGET_LAG` (retard toléré). Elles remplacent souvent le duo Streams + Tasks pour les pipelines de transformation.

© 2026 iAlexMG. Tous droits réservés.

Snowpipe contre `COPY`, la frontière d'examen — `COPY INTO` = chargement par lots, vous le déclenchez, il tourne sur votre entrepôt. Snowpipe = chargement continu, déclenché par l'arrivée des fichiers, serverless (pas d'entrepôt). Même moteur de chargement dessous, deux déclencheurs et deux modèles de coût.

Quatre écarts de plus, que l'examen oppose et qu'on ne devine pas :

	`COPY INTO`	**Snowpipe**
Historique de chargement	dans les métadonnées de la table, 64 jours, rendu par la commande	dans celles du pipe, 14 jours, à demander (REST, fonction de table, `ACCOUNT_USAGE`)
Transaction	une seule, pour tout le chargement	une ou plusieurs, selon le nombre et la taille des lignes
Ordre des fichiers	celui de la commande	pas de garantie : Snowpipe prend les plus anciens d'abord, sans promesse
Authentification	celle du client qui lance la commande	pour l'appel REST, une paire de clés avec un jeton JWT (*JSON Web Token*)

Piège — Snowpipe reconnaît un fichier déjà chargé à son nom et à son chemin, pas à son contenu. Un fichier corrigé puis redéposé sous le même nom ne sera pas rechargé, même si son contenu a changé. Et on ne charge pas le même jeu de fichiers par les deux voies : chacune tient ses propres métadonnées, et ce qui est chargé par l'une se recharge par l'autre.

7. La connectivité — relier un programme à Snowflake

Au-delà de Snowsight (Cours 01, section 3), un programme parle à Snowflake par :

- Pilotes (*drivers*) : Python, JDBC, ODBC, .NET, Go, Node.js, pour qu'une application se connecte et exécute du SQL.
- Connecteurs : Kafka (streaming d'événements → tables) et Spark (lire/écrire Snowflake depuis Spark) sont les deux classiques.
- API REST — la *SQL API* : soumettre du SQL par HTTP, sans pilote, pour l'automatisation.
- Storage integration (section 2) et PrivateLink (lien réseau privé vers Snowflake, sans passer par l'internet public, en Business Critical+). *(Reconnaître.)*

8. Les données non structurées

L'examen annonce qu'il teste l'usage des données structurées, semi-structurées et non structurées. Les deux premières sont au Cours 01 ; la troisième tient ici, parce qu'elle tient au stage.

Non structuré veut dire un fichier que rien ne range en lignes et en colonnes : un PDF, une image, un enregistrement audio, un format de métier. Snowflake ne le charge pas dans une table. Il le laisse où il est, dans un stage interne ou externe, et donne de quoi le lister et le lire.

© 2026 iAlexMG. Tous droits réservés.

Ce qui liste, c'est la directory table. Ce n'est pas un objet de base à part, mais une couche posée sur un stage, qui tient les métadonnées de ses fichiers : le chemin, la taille, la date de dernière modification, l'URL. Elle n'a pas de privilèges à elle, s'active à la création du stage ou plus tard, et s'interroge comme une table.

```
CREATE OR REPLACE STAGE docs DIRECTORY = (ENABLE = TRUE);
ALTER STAGE docs REFRESH;                        -- reprendre la liste des fichiers
SELECT relative_path, size, last_modified FROM DIRECTORY(@docs);
```

Ce qui donne accès au fichier, c'est une URL, et il y en a trois. L'examen les oppose :

URL	Ce qu'elle vaut	Comment on l'obtient
Scoped	temporaire : elle expire avec le cache de résultats, 24 heures aujourd'hui. Elle ouvre le fichier sans donner de droit sur le stage	`BUILD_SCOPED_FILE_URL`
File	permanente : il faut des droits sur le stage pour s'en servir, et un jeton d'autorisation pour l'appel REST	la directory table, ou `BUILD_STAGE_FILE_URL`
Pre-signed	ouverte : elle s'ouvre dans un navigateur, sans authentification, jusqu'à expiration de son jeton, dont on règle la durée	`GET_PRESIGNED_URL`

À retenir — Pour donner un fichier à quelqu'un d'autre, c'est la scoped URL : une vue peut la porter, et celui qui lit la vue atteint le fichier sans aucun droit sur le stage. La pre-signed URL, elle, n'exige rien du tout, pas même un compte Snowflake, et c'est exactement pour ça qu'elle se donne avec prudence.

QCM — style examen (Cours 04)

Comment répondre — Faites les dix questions avant d'ouvrir le corrigé, qui commence à la page suivante. Une question à choix multiple dit combien de réponses cocher, comme à l'examen. Comptez une minute par question.

1. Un chargement quotidien relance `COPY INTO ma_table FROM @stage` sur un stage où s'accumulent les fichiers des jours précédents.
    a. Les lignes d'hier entrent une seconde fois dans la table.
    b. La commande échoue, le stage contenant des fichiers déjà chargés.
    c. Seuls les fichiers neufs sont chargés : les métadonnées de charge écartent les autres.
    d. La table est vidée, puis rechargée depuis tout le stage.

© 2026 iAlexMG. Tous droits réservés.

2. Un fichier a été chargé par erreur avec le mauvais format. Vous voulez le recharger, tel quel, sans toucher au contenu déjà présent dans la table.
    a. `FORCE = TRUE` dans le `COPY`, qui passe outre les métadonnées de charge.
    b. `PURGE = TRUE`, qui remet les métadonnées à zéro après le chargement.
    c. `OVERWRITE = TRUE`, qui réécrit la table à partir du stage.
    d. `TRUNCATE TABLE` avant le `COPY`, pour que le fichier redevienne inconnu.
3. Un fichier de deux millions de lignes en porte douze mal formées. Vous voulez charger le reste et savoir combien ont sauté.
    a. `ON_ERROR = 'ABORT_STATEMENT'`, le défaut, qui liste les erreurs puis annule.
    b. `ON_ERROR = 'SKIP_FILE'`, qui saute les lignes fautives et garde le fichier.
    c. `VALIDATION_MODE = 'RETURN_ERRORS'`, qui charge et rapporte en même temps.
    d. `ON_ERROR = 'CONTINUE'`, qui charge le reste et rend le compte des lignes rejetées.
4. Un fournisseur vous envoie un fichier dont vous doutez. Avant d'écrire quoi que ce soit dans la table, vous voulez la liste de ce qui casserait.
    a. `ON_ERROR = 'CONTINUE'` : il écrit ce qui passe et signale le reste.
    b. `VALIDATION_MODE = 'RETURN_ERRORS'` : il joue le chargement sans rien écrire.
    c. Un `SELECT` sur le stage, qui écarte de lui-même les lignes invalides.
    d. `INFER_SCHEMA`, qui compare le fichier à la structure de la table.
5. *(choix multiple, sélectionnez-en DEUX)* Deux de ces affirmations séparent vraiment Snowpipe de `COPY INTO`.
    a. Snowpipe tourne sur l'entrepôt que vous lui désignez.
    b. Snowpipe se déclenche à l'arrivée des fichiers, et son calcul est serverless.
    c. L'historique de chargement de Snowpipe tient 14 jours dans les métadonnées du pipe, contre 64 jours pour `COPY` dans celles de la table.
    d. Snowpipe garantit l'ordre des fichiers, du plus ancien au plus récent.
6. Un fichier chargé par Snowpipe contenait une colonne erronée. Vous le corrigez et le redéposez dans le stage, sous le même nom.
    a. Snowpipe le recharge : il compare le contenu du fichier à chaque passage.
    b. Snowpipe le recharge une fois les 14 jours d'historique écoulés.

© 2026 iAlexMG. Tous droits réservés.

c. Snowpipe lève une erreur de doublon et suspend le pipe.

d. Snowpipe l'ignore : un fichier se reconnaît à son nom et à son chemin, pas à son contenu.

7. Une table d'agrégat doit suivre sa source avec au plus dix minutes de retard, et l'équipe ne veut rien avoir à orchestrer.

   a. Une Dynamic Table avec `TARGET_LAG = '10 minutes'`.

   b. Un Stream sur la source, lu par une Task toutes les dix minutes.

   c. Une vue matérialisée, à qui on fixe un horaire de rafraîchissement.

   d. Une Task seule, qui recalcule la table entière toutes les dix minutes.

8. Chaque analyste doit pouvoir déposer ses fichiers de travail, sans créer d'objet et sans déranger les autres.

   a. Un stage nommé interne par analyste, créé par l'administrateur.

   b. Le stage de table, `@%ma_table`, attaché à la table visée.

   c. Le stage utilisateur, `@~`, qui existe déjà pour chaque utilisateur.

   d. Un stage externe par analyste, vers un bucket personnel.

9. L'audit relève que le SQL qui crée vos stages externes porte les clés d'accès du bucket en clair.

   a. Ranger les clés dans le commentaire de l'objet, qui n'est pas lisible par tous.

   b. Créer une storage integration : elle garde les identifiants cloud, et le stage y fait référence.

   c. Poser une politique réseau qui restreint l'accès au bucket.

   d. Remplacer les stages externes par des stages internes, qui n'ont pas besoin de clés.

10. *(choix multiple, sélectionnez-en DEUX)* Des PDF de contrats vivent dans un stage. Une équipe doit pouvoir les ouvrir, sans recevoir de droit sur le stage. Deux affirmations sont exactes.

    a. Une directory table posée sur le stage liste les fichiers, avec leur chemin, leur taille et leur date.

    b. Il faut d'abord charger les PDF dans une table, en colonne `VARIANT`.

    c. Une scoped URL, portée par une vue, ouvre le fichier sans aucun droit sur le stage.

    d. La pre-signed URL est le bon choix ici, puisqu'elle expire avec la session de son auteur.

© 2026 iAlexMG. Tous droits réservés.

Corrigé expliqué — Cours 04

À lire après avoir répondu. Chaque réponse nomme la section à relire : ce que vous ratez ici vous dit quoi reprendre.

1. Réponse c. `COPY` garde la trace des fichiers déjà chargés, une soixantaine de jours, dans les métadonnées de la table : rejouer la commande ne crée pas de doublon et ne lève pas d'erreur, ce qui écarte les options a et b. Rien n'est vidé au passage non plus, contre l'option d : un `COPY` ajoute, il ne remplace pas (section 4).

2. Réponse a. `FORCE = TRUE` est fait pour ça : il ignore les métadonnées de charge et recharge le fichier. `PURGE` supprime les fichiers du stage après chargement, ce qui n'est pas ce que dit l'option b ; `OVERWRITE` sert au déchargement, contre l'option c. L'option d marcherait, et c'est le piège : `TRUNCATE TABLE` efface aussi les métadonnées de chargement, donc le fichier redeviendrait chargeable. Mais il vide la table, ce que l'énoncé interdit (section 4).

3. Réponse d. `CONTINUE` saute les lignes fautives, charge le reste, et la commande rend par fichier le nombre d'erreurs vues et la première d'entre elles. Le défaut `ABORT_STATEMENT` de l'option a annule tout dès la première erreur. `SKIP_FILE` de l'option b saute le fichier entier, pas les lignes. `VALIDATION_MODE` de l'option c n'écrit rien du tout (section 4).

4. Réponse b. `VALIDATION_MODE = 'RETURN_ERRORS'` joue le chargement à blanc et rend les erreurs, sans rien écrire : c'est l'essai avant l'ingestion. L'option a écrit ce qui passe, donc elle modifie la table. L'option c prête à un `SELECT` sur le stage un tri qu'il ne fait pas. `INFER_SCHEMA` de l'option d devine des colonnes à partir d'un fichier, il ne valide rien (section 4).

5. Réponses b et c. Snowpipe est continu et serverless, déclenché par l'arrivée des fichiers, et son historique de chargement est plus court que celui de `COPY`, 14 jours contre 64. L'option a décrit `COPY`, qui tourne sur votre entrepôt. L'option d promet un ordre que Snowpipe ne garantit pas : il prend les plus anciens d'abord, sans en faire une règle (section 6).

6. Réponse d. Snowpipe reconnaît un fichier à son nom et à son chemin : redéposé sous le même nom, un fichier corrigé ne repart pas, et c'est l'erreur classique en production. On le redépose donc sous un nom neuf. Les options a et b supposent une comparaison de contenu ou une péremption qui n'ont pas lieu, et l'option c invente une erreur que Snowpipe ne lève pas (section 6).

7. Réponse a. Une Dynamic Table est la voie déclarative : vous écrivez la requête cible et fixez le retard toléré, Snowflake se charge du reste. L'option b et l'option d demandent exactement l'orchestration que l'énoncé refuse. Une vue matérialisée, à l'option c, se maintient seule mais ne prend pas d'horaire et ne supporte pas les requêtes qu'une table de transformation demande (section 6).

8. Réponse c. Le stage utilisateur `@~` existe pour chaque utilisateur, sans rien créer, et il est invisible des autres : c'est exactement le besoin. L'option a fait créer un objet par personne. Le stage de table de l'option b est lié à une table précise. L'option d monte un bucket externe là où le stockage interne suffit (section 2).

9. Réponse b. La storage integration existe pour que le SQL ne porte plus de secret : l'objet range les identifiants du cloud, et le stage s'y réfère. L'option a déplace le secret sans le protéger. L'option c parle de réseau, pas d'identifiants. L'option d supprime le besoin plutôt que de le régler, et coupe l'accès au bucket du fournisseur (section 2).

© 2026 iAlexMG. Tous droits réservés.

10. Réponses a et c. La directory table est la couche qui liste les fichiers d'un stage, et la scoped URL donne accès à un fichier sans aucun droit sur le stage, ce qui en fait la forme normale du partage par une vue. L'option b confond non structuré et semi-structuré : un PDF ne se charge pas en `VARIANT`, il reste dans le stage. L'option d se trompe sur la pre-signed URL : elle n'exige aucune authentification et vit jusqu'à l'expiration de son jeton, ce qui en fait la plus ouverte des trois (section 8).

Points clés à retenir

- Le trajet : fichiers → stage → (format de fichier) → `COPY INTO` → table. `CREATE TABLE` fait la coquille (Cours 01) ; `COPY` la remplit.
- Stages : internes (utilisateur `@~`, table `@%`, nommé `@stage`) ou externes (S3/ Azure/GCS via storage integration, qui évite les clés en clair). `PUT`/`GET` déposent/récupèrent.
- `COPY INTO <table>` (chargement par lots) : `ON_ERROR` (`ABORT_STATEMENT` défaut / `CONTINUE` / `SKIP_FILE`), `VALIDATION_MODE` (test à blanc), idempotent (fichiers déjà chargés ignorés, sauf `FORCE`). `COPY INTO <emplacement>` = déchargement.
- Ingestion continue : Snowpipe (continu, serverless, à l'arrivée des fichiers) ; Snowpipe Streaming (lignes, faible latence) ; Streams (CDC) ; Tasks (planification / DAG) ; Dynamic Tables (transformation déclarative, `TARGET_LAG`).
- Snowpipe contre `COPY` : historique de chargement 14 jours contre 64, une ou plusieurs transactions contre une seule, aucun ordre garanti, et un fichier reconnu à son nom, pas à son contenu.
- Non structuré : le fichier reste dans le stage ; la directory table le liste ; trois URL le donnent à lire, scoped (temporaire, sans droit sur le stage), file (permanente, avec droits) et pre-signed (ouverte, sans authentification).
- Connectivité : pilotes (Python/JDBC/ODBC/.NET/Go/Node), connecteurs (Kafka, Spark), SQL API (REST), storage integration, PrivateLink (réseau privé, Business Critical+).

© 2026 iAlexMG. Tous droits réservés.